ORIGINAL ARTICLE

GY-YOLO: ghost separable YOLO for pedestrian detection

Ali M. Elhenidy[1,2] · Labib M. Labib[1] · Amira Y. Haikal[1] · Mahmoud M. Saafan[1,2]

Received: 17 February 2024 / Accepted: 19 March 2025 / Published online: 16 May 2025
© The Author(s) 2025

Abstract
In recent years, there has been impressive development in human detection. The main challenge in pedestrian detection is the training data. To assess detectors in crowd scenarios more effectively, a novel dataset in this study called the HEP dataset (Hybrid Egyptian Pedestrian dataset) is introduced. The HEP dataset is extensive, has comprehensive annotations, and is highly diverse. The dataset images are collected by two different means. Most of the images are collected from different mobile cameras for people crossing the street in high crowded streets in Egypt, and the rest of the images are collected from the web. That is why the dataset is called hybrid. The collected dataset is more suitable for pedestrian detection as the whole images focus on pedestrian scenarios for people outdoors crossing the street. This outperforms the previous benchmarks such as CrowdHuman and WiderPerson which collect data from the web and surveillance cameras with lots of images for indoor people. GS-YOLO also is proposed to address the real-time performance and the occlusion in the crowd scenes issues. GS-YOLO is a novel pedestrian detection model that utilizes efficient Ghost and depth separable convolution modules. GS-YOLO replaces all the convolution layers in the backbone and the head of the original YOLOv8 with Ghost and depth separable modules, respectively. A deformable to-features module is proposed to enrich features for the different feature pyramid networks. GS-YOLO is trained and tested over the collected dataset and other benchmarks like CrowdHuman and WiderPerson datasets. GS-YOLO achieves competitive results over the state-of-the-art models such as YOLOv5 and YOLOv8. GS-YOLO achieves 92.8% mAp on the HEP dataset, while YOLOv5 achieves 90.3% mAp and YOLOv8 achieves 91.1% mAp.

Keywords Pedestrian detection · Deep learning · Computer vision · YOLO

1 Introduction

Pedestrian detection is considered a branch of general object detection tasks that concerns mainly detecting people outdoors. As a result, pedestrian detection is involved in many applications self-driving cars [1], video surveillance [2], and autonomous robots [3]. One of the potential advantages of pedestrian protection systems (PPSs) applying pedestrian detection in self-driving cars is to decrease the number of car-to-pedestrian accidents on the road. Pedestrian detection is a very challenging problem due to the highly dynamic environment and the lack of specific pedestrian datasets in traffic scenarios.

According to [4], pedestrian detectors do not generalize well in autonomous driving applications due to the limited diversity of the existing dataset sources and do not make use of YouTube or Facebook. CrowdHuman [5] and WiderPerson [6] collect images from the web and surveillance cameras. These datasets cover diverse scenarios of crow of humans but are not efficient for autonomous driving tasks as less attention to the traffic

environment is paid as shown in Fig. 1. These datasets contain many scenarios of indoor crowding or inside public places such as schools, playgrounds, and dance parties. Wider Pedestrian [7] and TJU-DHD [8] address this limitation and collect data specific to self-driving cars by collecting images from surveillance cameras or cameras located on driving vehicles through regular traffic in urban environments.

Pedestrian detection is a difficult problem for original object detectors as they provide duplicate detections for the same thing due to the overlapping objects, as the pedestrian frequently appears in crowded situations. The handcrafted NMS has recently been abandoned by end-to-end detectors [9, 10], which now treat object detection as a direct set prediction problem. These end-to-end techniques, however, perform much worse in pedestrian identification and crowd scenarios [11]. To address these shortcomings, PEDETR [11] adapts the DETR underlying architecture for pedestrian detection. To precisely control the attention positions, PEDETR suggests a new decoder module called decoder with dense queries and rectified attention field (DQRF), which alters the cross-attention layer.

DETR-based models achieve high accuracy but are still limited due to the computational complexity and the real-time shortage. YOLO-based pedestrian detection models are effective because of their quick inference speed, which makes them appropriate for real-time applications. The feature extraction backbone of YOLOpeds [12] is made up of ten separable convolution layers, which are more effective than conventional convolution layers while

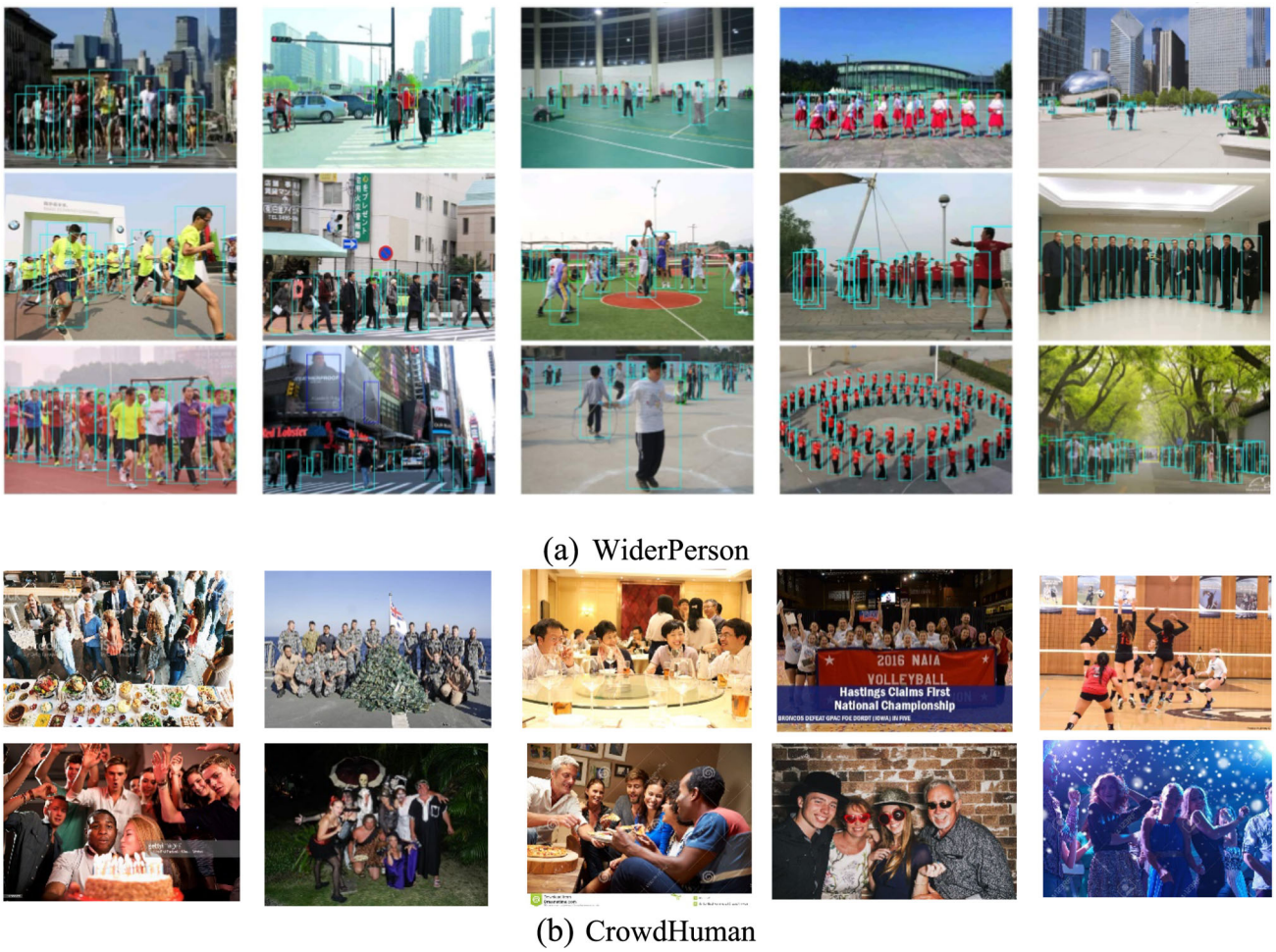


Fig. 1 Samples of WiderPerson (**a**) and CrowdHuman (**b**) datasets that collect data by crawling the web and through surveillance cameras. The images explore people in different crowded events with less attention to traffic scenarios which is not efficient for autonomous driving applications

using fewer parameters. Traditional approaches have a high false detection rate for pedestrian targets that are modest in size, such as roads. Gu et al. [13] significantly enhanced the YOLOv5 method to solve this issue. They reduced the missed detection rate of tiny outdoor targets by enhancing the feature pyramid network structure to enable multi-scale feature fusion for dense objects and combining it with a layer specifically designed for small target detection. To address the small pedestrian ratios the RSA-YOLO (ratio-and-scale-aware YOLO) [14] is introduced. The RSA-YOLO model suggests novel scale-aware techniques that use multiresolution fusion to address the issue of pedestrians in photos being mistakenly identified as being noticeably small.

In their creative modifications to the YOLOv8 backbone network, Lou et al. [15] recommended using cascaded depth-separable convolutions rather than the conventional C2F feature fusion module. This modification enhances the network's capacity to identify tiny pedestrian targets in sensor devices by preserving a greater variety of multi-scale pedestrian information. YOLOv5s-G2 [3] minimizes the complexity of the YOLOv5s network without losing precision, by applying Ghost and GhostC3 modules. They can attain a balanced proportion of portability and precision. The occlusion issue is discussed in [16] by introducing the Feature fusion module (FFM) to the base model of YOLOv5. FFM fuses the whole pedestrian features with the local head and leg features to finally predict the pedestrian bounding box. However, this technique adopts a fixed fusion ratio of the head and the leg with respect to the pedestrian dimensions which is not suitable for all cases such as different scales or image resolution.

Modern methods primarily rely on substantial computational resources, particularly graphics card acceleration, to attain optimal accuracy and processing speed, even with the ongoing advancements in pedestrian detection approaches. Pedestrian recognition in dense or small-target situations is impacted by these algorithms' propensity to overlook fine-grained local information in favor of multi-scale feature fusion and readily discernible global features. This study presents GS-YOLO, a novel lightweight dense pedestrian identification method based on YOLOv8n, which aims to solve these challenges and achieve a balance between detection accuracy and speed, particularly in complicated contexts like street crossings. GS-YOLO utilizes efficient Ghost and depth separable convolution modules. GS-YOLO replaces all the convolution layers in the backbone and the head with Ghost and depth separable modules, respectively. A deformable to-features (D2f) module is proposed to enrich features for the different feature pyramid networks. D2f module replaces the traditional convolution layer in original c2f introduced in YOLOv8 by a deformable convolution one. Deformable Convolutions enable the receptive field to dynamically adjust to the shapes and orientations of objects. Scaling, rotation, translation, and perspective distortions are the spatial transformations the D2f layer can manage. A Haar Cascade Fusion (HCF) module also is proposed to fuse the local face features with the lower body features to overcome the occlusion issue. These features fusion is used to predict the final pedestrian bounding box.

In this paper, a novel hybrid pedestrian dataset is introduced to enrich the pedestrian dataset and release more suitable data that concerns only pedestrian situations. The dataset is collected from mobile cameras for crowded crosses in different cities in Egypt and images from the web for people in traffic scenes. The collected images have diverse resolutions, the view angle (images captured from a camera installed in a moving vehicle and a mobile camera from the roadside), and the image sources (live cross intersections in different cities in Egypt and web images). The proposed dataset is trained and tested over the recent state-of-the-art object detector yolov5 [17] and YOLOv8 [18].

The main contribution of this paper is listed below:

- Propose a novel object detection model called GS-YOLO for pedestrian detection which is a light-weight model that achieves a balance between accuracy and processing speed.
- Propose a Haar Cascade Fusion (HCF) module to overcome the occlusion problem by fusion of the face and lower body features to get the final prediction.
- Collect a novel pedestrian dataset called the Hybrid Egyptian Pedestrian dataset (HEP) that covers more suitable situations for pedestrian detection.

The rest of the paper is organized as follows: Sect. 2 discusses the background and the recent terminologies and techniques used in object detection. Section 3 illustrates the proposed model including the components in each stage. The HEP dataset will be covered in Sect. 4 as well as the experiment results, and the future work will be in Sect. 5. Finally, the conclusions are discussed in Sect. 6.

2 Background and historical review

2.1 Object detection categories

Object detection is one of the fundamental computer vision tasks, concerned with detecting where the objects are located in the image and identifying which categories they belong to, as shown in Fig. 2. Object detection has witnessed a great advance in the last decade due to the big data era we live in and the powerful computing tools researchers have. Object detectors can be classified into six different categories as shown in Fig. 3 and will be discussed in the following sections. Two recent surveys [19, 20] discussed the latest object detection models and techniques. In this section, we will discuss the different object detection models from the classic hand-crafted detectors to the transformer-based models.

2.1.1 Hand-crafted detectors

Early attempts at object detection back to 1986 when John Canny [21] proposes a computational and handcrafted approach to detect edges. 21 years ago, Viola Jones Detector [22] was proposed to achieve real-time human face detection using the sliding windows technique to go through all possible locations in the image and search for the human face. In 2005, HOG [23] was proposed as a feature descriptor that follows a sequence of hand-crafted feature extraction processes as shown in Fig. 4 and then is forwarded to SVM to get the final prediction.

Similar to SIFT [24], HOG counts occurrences of gradient orientation in the localized portion of an image. After achieving good results on the MIT pedestrian dataset, the authors decided to create a new challenging set containing 1800 pedestrian images with a large range of backgrounds and different poses.

2.1.2 Two-stage object detection

In 2014, Girshick et al. influenced object detection by proposing R-CNN [25] (Region with CNN) which has a great impact on the object detection community. R-CNN [25] is the first paper to adopt CNN to object detection and combine it with the classic region proposals method (i.e., selective search [26]). CNN backbone extracts features from the 2000 region proposal and then is fine-tuned to get the final prediction.

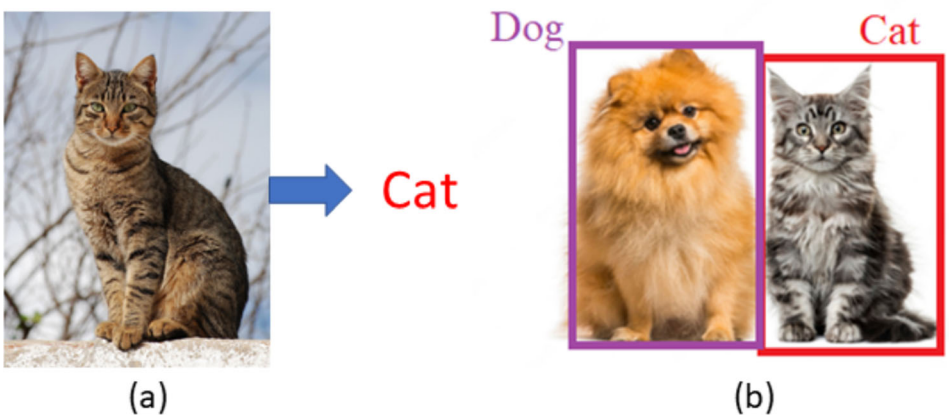


Fig. 2 The difference between image classification (**a**) and object detection (**b**)

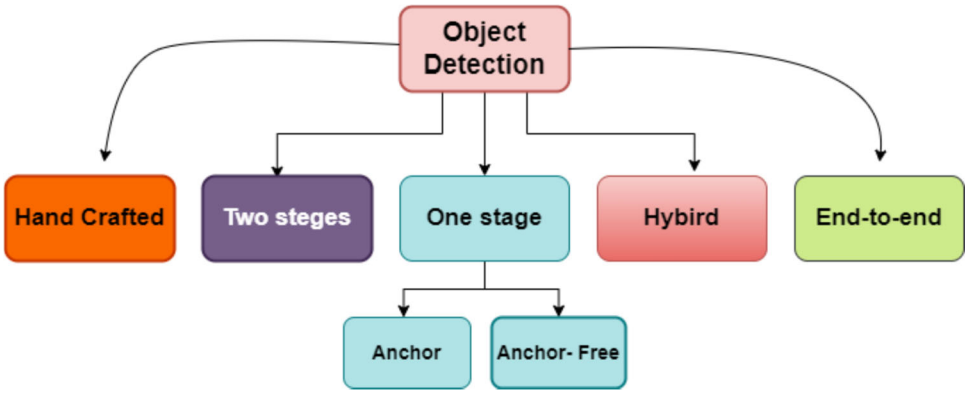


Fig. 3 Object detectors categories

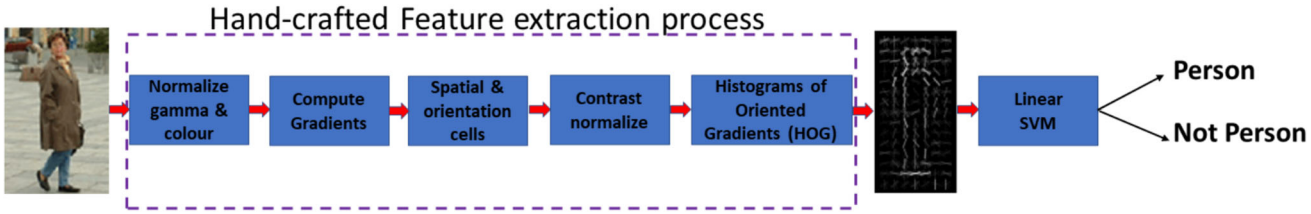


Fig. 4 HOG architecture

SPP-NET [27] generates the region proposal module in a separate branch and passes the entire image to the CNN to extract features from the image only once. SPP layer on top of the last convolutional layer maps the region proposal to the feature map and obtains features of fixed length for the FC layers. This technique led to 24–102 × faster than R-CNN because it only needs to run the convolutional layers once on the entire image. Fast R-CNN [28] proposed a single-stage training algorithm that jointly learns to classify object proposals and refine their spatial locations using one single backpropagation path.

Faster RCNN [29] extracts region proposals using a fully convolutional network (FCN) and introduces a Region Proposals Network (RPN) that replaces the traditional Selective search [26] technique. Adopting the fully convolutional technique in the region proposal has the advantage of sharing convolutional features with the detection network which is Fast RCNN [28] and enabling nearly cost-free region proposals.

2.1.3 One-stage object detection

In 2016, W. Liu et al. proposed SSD [30] a single shot detector for multiple categories is considered to be the first deep one-stage free anchor detector by eliminating bounding box proposals and pixel or feature resampling stage and performing detection using a single deep neural network. R. Joseph et al. proposed a new single regression novel called YOLO [31]. YOLO network inspired a series of improvements [17, 32–34] that will be discussed in detail in Sect. 3.1.

Anchor boxes introduced in [29, 32, 33] suffer from some drawbacks:

- A large set of anchors is needed for training that slows down training.
- Only a few of the suggested anchors sufficiently overlap with the ground truth which creates a huge imbalance between positive and negative anchor boxes.
- Many hyperparameters and design choices make the process more complicated.

CornerNet [35] proposes an anchor-free object detector that detects bounding boxes as a pair of key points—top left and bottom right corner—using a single convolutional network. CornerNet predicts a heatmap for the top left corners of all the same object categories and another heatmap for the bottom right and an embedding vector is used to group pairs that belong to the same object.

CenterNet [36] modifies CornerNet [35] by exploring the visual pattern in bounding boxes introducing center pooling and cascade center pooling modules to enrich recognizable information. The score of the bounding boxes will be calculated by the average scores of three points rather than two, i.e., the top left corner, the bottom right corner, and the center keypoint. The center key point is used to reduce the number of incorrect bounding boxes if there are no center key points detected in the bounding box central region.

2.1.4 Hybrid detection

Some architectures combine the advantages of two-stage detectors, i.e., high accuracy, and the advantages of one-stage detectors, i.e., high efficiency, and propose a new architecture that overcomes both like RefineDet [37].

RefineDet [37] introduced a new architecture that consists of two modules which are the Anchor refinement Module (ARM) and Object Detection Module (ODM). Anchor refinement Module (ARM) is constructed on top of VGGNet [38] and ResNet [39] after removing the classification layer to reduce the search space for classification by removing the negative anchors and adjusting the locations and sizes of anchors feed to the regressor. Object Detection Module (ODM) is the regressor module that creates corresponding bounding boxes and labels. The link between Arm and ODM is done by a transfer connection block (TCB) that converts features from ARM to ODM.

2.1.5 End-to-end detection

DETR [9] was the first paper to introduce Transformers attention-based method used for machine translation task—to object detection task predicting a set of detections directly end to end by removing hand-crafted modules such as non-maximum suppression and anchor module.

DETR [9] suffers from low performance and slow convergence problems due to the attention module that casts uniform weights to all pixels and needs long training epochs to focus on meaningful locations. Deformable DETR [10] solves this limitation by adopting deformable convolution [40] which attends to only a small set of sampling points around a reference point.

2.2 Light-weight convolutional networks

2.2.1 Depthwise separable convolutions

The core of CNN is the convolution layer which mainly is a matrix convolution operation between the kernel or the filter and a small portion of the image as shown in Fig. 5. MobileNet [41] introduces depthwise separable convolutions instead of regular convolutions rather to reduce the number of parameters and computations. Depthwise separable convolutions divide the convolution operation into two steps: depthwise convolution and pointwise convolution as shown in Fig. 6. The depthwise convolution stage splits the kernel volume into several two-dimensional kernels equal to the number of channels for the input image. The pointwise convolution

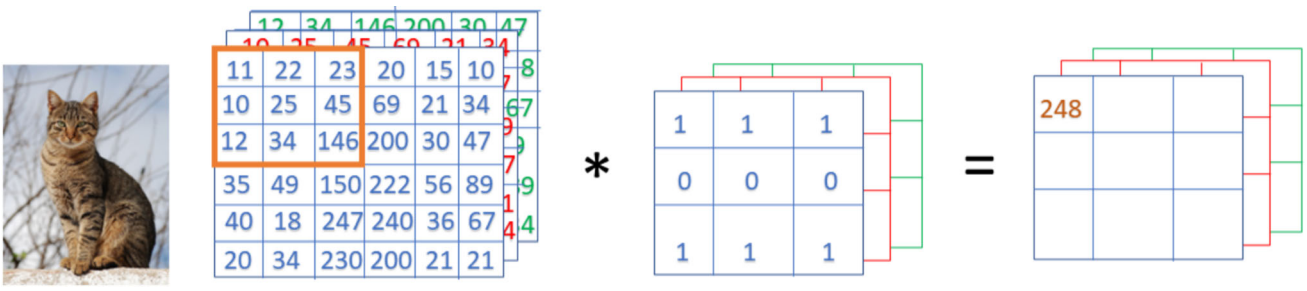


Fig. 5 Convolution operation

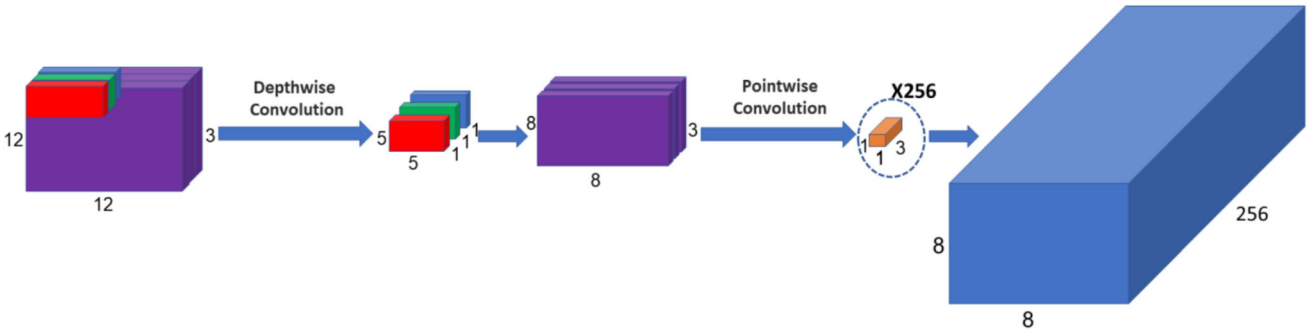


Fig. 6 Separable convolutions

uses a 1 × 1 kernel that has a depth of the same input image channels that iterates through every single point. This kernel is repeated for desired output features depth or channels. All the computation over the depthwise separable convolutions module is performed on tensors of a single depth that will reduce the computations by a significant factor.

2.2.2 Ghost module

Considering the frequent duplication in intermediate feature maps produced by standard convolutional networks, GhostNet [42] is introduced to suggest using fewer of the convolution filters that are needed to generate them. The ghost module removes the bias term from the convolution calculations and applies linear computations to obtain the intermediate features. Intrinsic features are calculated using a sequence of linear operations, as shown in Fig. 7. Ghost module proves its efficiency in pedestrian detection tasks based on YOLO architecture [3].

2.2.3 Deformable convolution

To address the model geometric transformation problems with convolutional neural networks (CNNs), deformable convolution [40] was initially developed. The constant receptive field of the previous activation unit is deformed to account for the scale of different objects and has different receptive fields depending on the object's scale by adding 2D offsets to the regular grid sampling locations in the standard convolution operation. Additional convolutional layers can be used to learn the added offsets from the preceding feature maps. Consequently, the employed deformation is dense, local, and feature-adaptive.

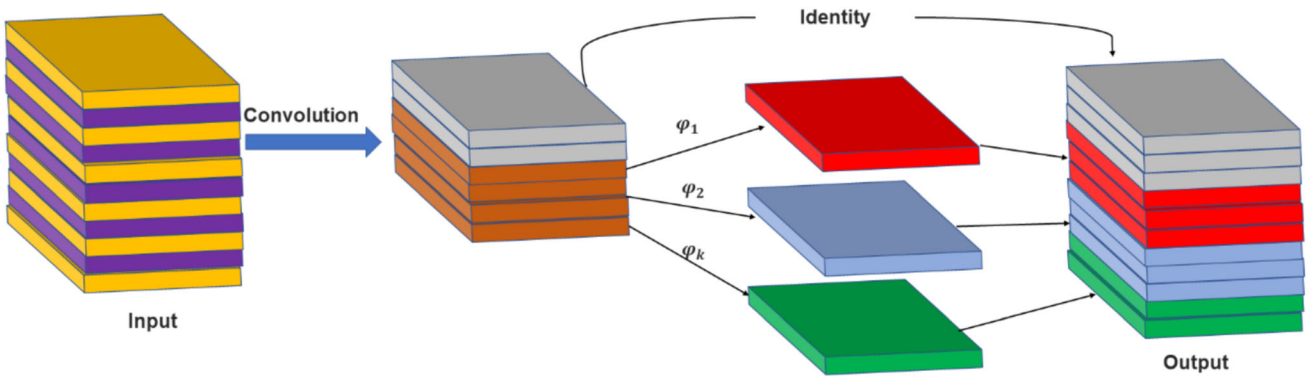


Fig. 7 Ghost module

2.3 Related work of object detection

2.3.1 YOLO

YOLO [31] network divides the input image into a grid of SxS. If there is an object and its center falls into a grid cell, that grid cell is responsible for detecting that object. Then, bounding boxes are predicted and the corresponding confidence scores are calculated which reflects how confident this box contains an object or not. Using the IOU threshold, a conditional class probability is calculated for boxes that claim to be an object. YOLO architecture—inspired by the GoogLeNet [39] model—consists of 24 convolutional layers—for Fast YOLO they use 9 CNNs instead of 24— followed by 2 fully connected layers.

YOLOv2 [32] was proposed to achieve improvements on the previous YOLO [31] version in terms of accuracy—Batch Normalization [43], High-Resolution classifier, Convolutional with Anchor boxes, Dimension Clusters, Direct location prediction, Fine-Grained features, Multi-Scale training—and speed by proposing Darknet-19 architecture that requires only 5.58 billion floating-point operations to process an image instead of VGG-16 [38] that require 30.69 billion fp operations and GoogLeNet [39] with 8.52 billion operations, and able to manage the tradeoff between these two terms. Combining these improvements, YOLOv2 introduced YOLO9000, a real-time object detection system, that can detect over 9000 object categories using a joint training algorithm that allows training on both detection and classification data.

YOLOv3 [33] makes some extra effort to improve YOLOv2 by simply implementing good ideas from previous work like FPN [44] and ResNet [39] and then comparing results, in terms of speed and accuracy, with state-of-the-art object detection such as SSD variants [30]. YOLOv3 uses logistic regression instead of the sum of squared errors to predict an objectness score of each bounding box, which leads to assigning one bounding box before each ground truth object. Instead of using softmax for class prediction, YOLOv3 uses binary cross-entropy to enable multilabel classification when dealing with more complex datasets like the Open Images dataset [45]. Inspired by FPN [44], YOLOv3 predicts boxes at 3 different scales by adding several convolutional layers to upsample feature maps by 2 $\times$ to get more powerful semantic information then uses k-means clustering to detect bounding boxes. Introducing Darknet-53 architecture as a feature extractor, YOLOv3 is 1.5 $\times$ faster than ResNet-101 [39], and 3 $\times$ faster than SSD [30]. YOLOv3 improves accuracy for detecting small objects, but on the other hand, has a poor performance for medium and larger size objects. Another limitation of YOLOv3 is that it has poor AP on the COCO [46] 0.5-0.9 IOU metric, but it is good on the old 0.5 IOU.

YOLOv4 [34] is proposed to enable training neural networks only on one GPU in real-time scenarios accurately and very fast using parallel computations rather than BFLOP. Introducing new features, e.g., weighted residual connections (WRC), cross-stage partial connections (CSP), cross mini–Batch Normalization (CmBN), Self-adversarial training (SAT), Mosaic data augmentation, and Mish-activation to increase the accuracy of CNN and make it general for most of dataset and detection algorithms.

YOLO v4 chooses CSPDarknet53 as a backbone rather than CSPDarknet50 as it is more suitable for detection. On Top of the backbone, they use SPP-NET [27] as an additional module to increase respective field and extract significant features and PANet [47] is used as a path aggregation neck to collect feature maps from different stages rather than FPN [44] and use YOLO v3 [33] as the Head of the YOLO v4. YOLO v4 uses a Genetic Algorithm search for optimal hyperparameters during the training phase and introduces a new data augmentation method, e.g., Mosaic and Self-Adversarial training.

YOLOv5 [17] was developed by Ultralytics and released in 2020. YOLOv5's backbone is CSP-Darknet53. CSP-Darknet53 is essentially the Cross Stage Partial (CSP) network approach applied to the convolutional network Darknet53, which served as the foundation for YOLOv3. YOLOv5 significantly altered the model neck in two ways. Initially, a version of Spatial Pyramid Pooling (SPP) was employed, and the BottleNeckCSP was included in the design of the Path Aggregation Network (PANet) [47].

The most recent and advanced YOLO model, YOLOv8 [18], applies to tasks like instance segmentation, object detection, and image classification. A C2f module now makes up the model's framework rather than a C3 one

used in YOLOv5. Where C3 and C2 are CSP (Cross Stage Partial) bottlenecks with 3 and 2 convolutions, respectively, C2f is a faster implementation of C2. The model concatenates the output of each bottleneck module in C2f, which is how the two differ from one another. On the other hand, the model in C3 takes advantage of the last bottleneck module's output.

2.3.2 Pedestrian detection

Pedestrian detection is a challenging task for original object detectors for many reasons. As the pedestrian often shows in crowd scenes, most object detectors produce duplicate detection for the same object because of the overlapping objects. IterDet [48] proposes an iterative scheme to detect a new subset of objects at each iteration. Bounding boxes from all iterations are considered to avoid duplicate detection for the same object. This technique is applied to two-stage and one-stage detectors which apply the NMS to score and remove duplicate detections. Recently, end-to-end detectors [9, 10] have gotten rid of the handcrafted NMS and approach object detection as a direct set prediction problem. However, these end-to-end methods perform very badly in pedestrian detection and crowd scenes even worse than classical two-stage methods [11]. PEDETR [11] discusses these drawbacks and modifies the DETR base architecture to be suitable for pedestrian detection. PEDETR proposes a new decoder module called decoder with Dense Queries and Rectified Attention Field (DQRF), which modifies the cross-attention layer to accurately tune the attention positions. Another recent work [49] addresses this issue and proposes a progressive method to select queries with high confidence scores as accepted queries and then prone them to generate true positive predictions.

Pedestrian detection models based on YOLO are efficient due to its fast inference speed which is suitable for such real-time applications. YOLOpeds [12] consists of ten separable convolution layers as a feature extraction backbone that are more efficient with a reduced number of parameters than the traditional convolution layers. To address the small pedestrian ratios the RSA-YOLO (ratio-and-scale-aware YOLO) [14] is introduced. The RSA-YOLO model suggests novel scale-aware techniques that use multiresolution fusion to address the issue of pedestrians in photos being mistakenly identified as being noticeably small. Using intelligent splits to appropriately and repeatedly clip photos is suggested to assist RA-YOLO in extracting pedestrian detection information from images with different resolutions.

In [16], the occlusion problem is addressed by adding the Feature Fusion Module (FFM) to the YOLOv5 basic model. To ultimately forecast the pedestrian bounding box, FFM combines the local head and leg features with the entire pedestrian features. Another study [50] discusses the occlusion problem by introducing a segmentation function that can divide pedestrians who do not overlap in one image into two sub-images added to the proposed model. To produce the final detection result, multiresolution adaptive fusion was applied to the output of each picture and subimage using a network architecture.

For pedestrian targets that are small in size, like roads, traditional methods have a high false detection rate. To address this problem, YOLO-SSFS [13] greatly improved the YOLOv5 approach. By adding a layer especially made for small target identification and improving the feature pyramid network topology to allow multi-scale feature fusion for dense objects, they decreased the missed detection rate of small outdoor targets. Instead of using the traditional C2F feature fusion module, Lou et al. [15] suggested employing cascaded depth-separable convolutions in their innovative alterations to the YOLOv8 backbone network. By maintaining a wider range of multi-scale pedestrian information, this change improves the network's ability to detect small pedestrian targets in sensor devices. YOLOv5s-G2 [3] uses Ghost and GhostC3 modules [42] to reduce the YOLOv5s network's complexity without sacrificing accuracy. They can achieve an equilibrium between portability and accuracy. YOLOv8-CB [51] enhances the characterization of multiscale feature semantics and location information by introducing a lightweight cascade fusion network, CFNet (cascade fusion network), and a CBAM attention module. It also overlays a bidirectional weighted feature fusion path BIFPN structure to fuse more efficient features and enhance pedestrian detection performance. A summary of the pedestrian detection models is listed in Table 1.

Table 1 Summary of the pedestrian detection models

Paper	Backbone	Contribution	Pros	Cons
IterDet [48]	DETR	Propose an iterative scheme to detect the object at the current iteration based on all the previous iterations	Ensure that the same object is not detected twice	Need a post-processing method to remove duplicates in every iteration
PEDETR [11]	DETR	Propose a novel decoder with dense queries and rectified attention field and design a heuristic improvement of KM algorithm in the bipartite matching of Encoder Decoder	Handle the poor performance issue of the Encoder Decoder	The anticipated bounding boxes are less likely to change in the later layers
Zheng et al. [49]	DETR	Propose a progressive predicting method by selecting accepted queries prone and refining the rest noisy queries according to the previously accepted predictions	Discuss the performance saturation of the decoding stage when the depth increases	Difficult to deploy on devices with limited computing capacity
YOLOpeds [12]	YOLO	Based on separable convolutions, a computationally efficient architecture suggests combining multi-scale feature fusion with dense connections between layers	Reduce the number of parameters and operations while increasing representational capacity	Can't generalize to high-resolution images and lacks high accuracy
RSA-YOLO [14]	YOLOv3	Adjust the length and the width of the input layer of YOLOv3 using ratio-aware mechanisms. Also, combine high-resolution input images with two local images split from the original one	Handle the small pedestrian ratios and the difference in the aspect ratio of the input image	Low performance in occluded situations
Hsu et al. [50]	YOLOv4	Propose a segmentation function capable of creating two sub-images from pedestrians in a single image that does not overlap	Solve the uncertainty of the different scales and occluded pedestrian modes	Hyperparameters must be manually chosen, and the segmentation function's operation causes the inference time to increase
YOLO-SSFS [13]	YOLOv5	Replace the pooling layer in YOLOv5 with a proposed CNN block called SPD-Conv. Also, Complete-IoU (CIoU) is replaced by the SIoU loss to speed up the convergence of the loss function	Improve multi-scale fusion, reduce the information loss, and improve the learning ability	Failed to generalize to non-motorized vehicles and pedestrians
DC-YOLOv8 [15]	YOLOv8	Replace the C2F module in the YOLOv8 with a proposed DC module which stacks depth-wise separable convolution and ordinary convolution	High precision and accuracy for small object detection	Requires a lot of processing power and overlooks fine-grained local information, which has an impact on pedestrians for small targets
YOLOv5s-G2 [3]	YOLOv5	Incorporate Ghost, GhostC3, and Global Attention Mechanism (GAM) modules to YOLOv5s network	Minimize computational cost during feature extraction and improves feature extraction accuracy	The complexity of the network and not easily deployed on resource-limited devices
Luo et al. [16]	YOLOv5	Introduce the Ghost module and SE block to the original YOLOv5 model. A local feature fusion module (FFM) is also introduced	Address the occlusion issue and propose a lightweight version of YOLOv5	The local fusion is limited to a fixed aspect ratio and image resolution
YOLOv8-CB [51]	YOLOv8	Introduce a lightweight cascade fusion network, CFNet (cascade fusion network), and a CBAM attention module	Improve the characterization of multiscale feature semantics and location information,	Low performance in dense environments where severely occluded pedestrians with small targets

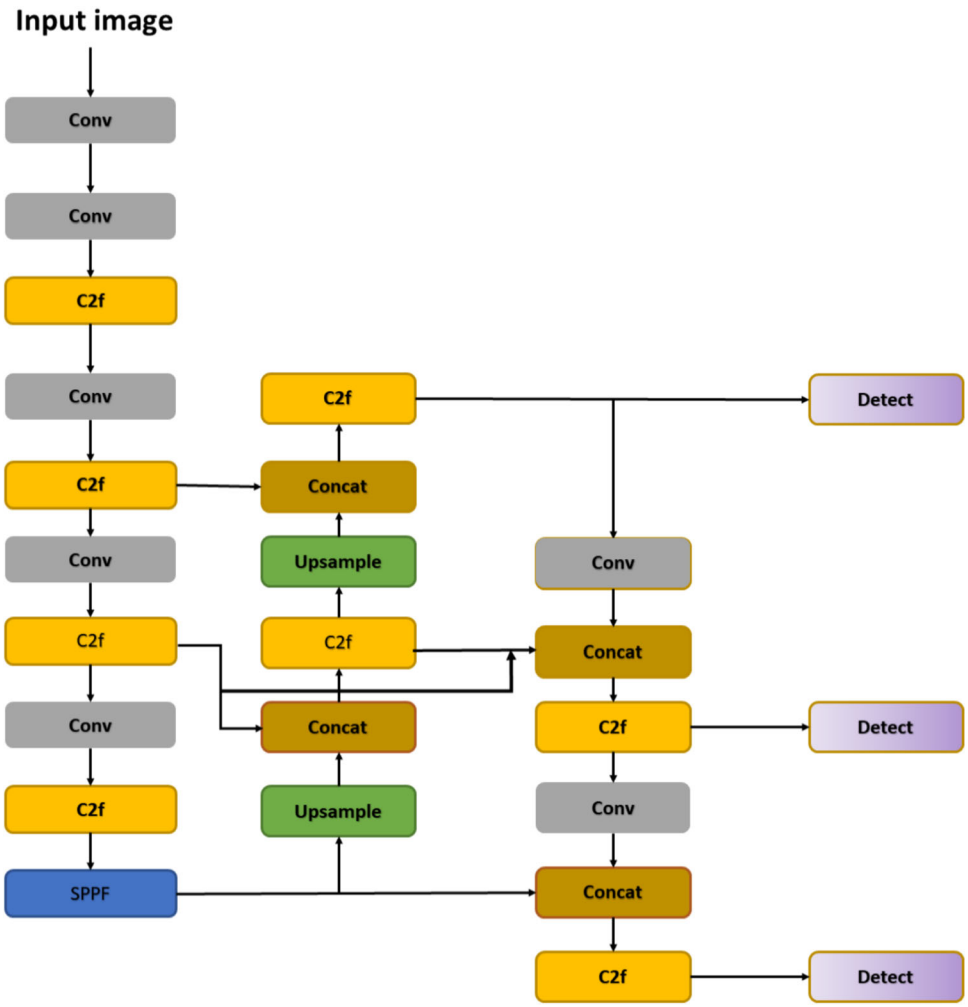


Fig. 8 Original YOLOv8 architecture

3 Proposed model

3.1 Ghost separable YOLO (GS-YOLO) architecture

To guarantee accuracy with less algorithmic complexity than the original YOLOv8 (Fig. 8) network and effectively reduce the requirement for computational power, the GS-YOLO network, a lightweight pedestrian detection network, is introduced based on the YOLOv8m network. GS-YOLO starts with YOLOv8m as a base line replacing the convolution layers in the backbone to the Ghostconv module to decrease the number of the parameters and the floating point operations. Ghost modules are designed to further reduce the complexity of the model. Following the original YOLOv8 work, the C2f module is adopted in the backbone which is a faster implementation of the C2 module used in YOLOv5. The C2 module is a CSP bottleneck with 2 convolutions, while C2f is a faster implementation of C2 adopting the residual connections.

In the neck part, the PAN-FPN is adopted to fuse information at different scales to strengthen the fusion. Two upsampling processes are formed from multiple C2f modules and are combined with the final decoupled head which is the D2f layer. In the head, Bi-FPN confusion is adopted to enrich the extracted features. The down-sampling process in the head replaces the traditional convolution layer with depthwise separable convolutions for efficient and fast computations. In the training phase, DW convolution uses only one-third of the computations and has a smaller volume than standard convolution. Every researcher is willing to employ regular convolution for downsampling since small convolution kernels can extract little information.

3.2 D2f module

Before the three detect layers, the D2f layer is introduced to enhance the geometric transformation modeling capacity. By learning offsets for convolution kernels, Deformable Convolutions enable the receptive field to dynamically adjust to the shapes and orientations of objects. Scaling, rotation, translation, and perspective distortions are the spatial transformations the D2f layer can manage. The model's generalization is enhanced by adjusting to the geometric fluctuations in the input data, particularly for the crowded and unpredictable situations of pedestrian detection. The difference between the three model structures (C2, C2f, and D2f) is shown in Fig. 9.

3.3 Haar Cascade fusion

To overcome the occlusion issue the Haar Cascade Fusion (HCF) module is proposed. HCF adopts the same strategy in the Feature Fusion Module (FFM) [16], but the head and leg features are extracted from the Haar Cascade classifier [22]. HCF utilizes the hand-crafted Haar Cascade face and leg detection rather than the costly annotated ground truth and the complex computations of the propagation through the YOLO network. Haar Cascade features in used to resort and construct the final prediction. HCF returns the pedestrian's head and leg areas to the pedestrian's overall box by the human body's natural proportions according to Formula 1 as shown in Fig. 10.

$$xc = x_{\text{chead}},\ \mathrm{y_c} = \mathrm{y_{chead}} + 2\mathrm{H}$$

$$W = 2W_{\text{head}},\ H = 5H_{\text{head}}$$

$$xc = x_{\text{cleg}},\ y_c = y_{\text{cleg}} - \frac{1}{2}H$$

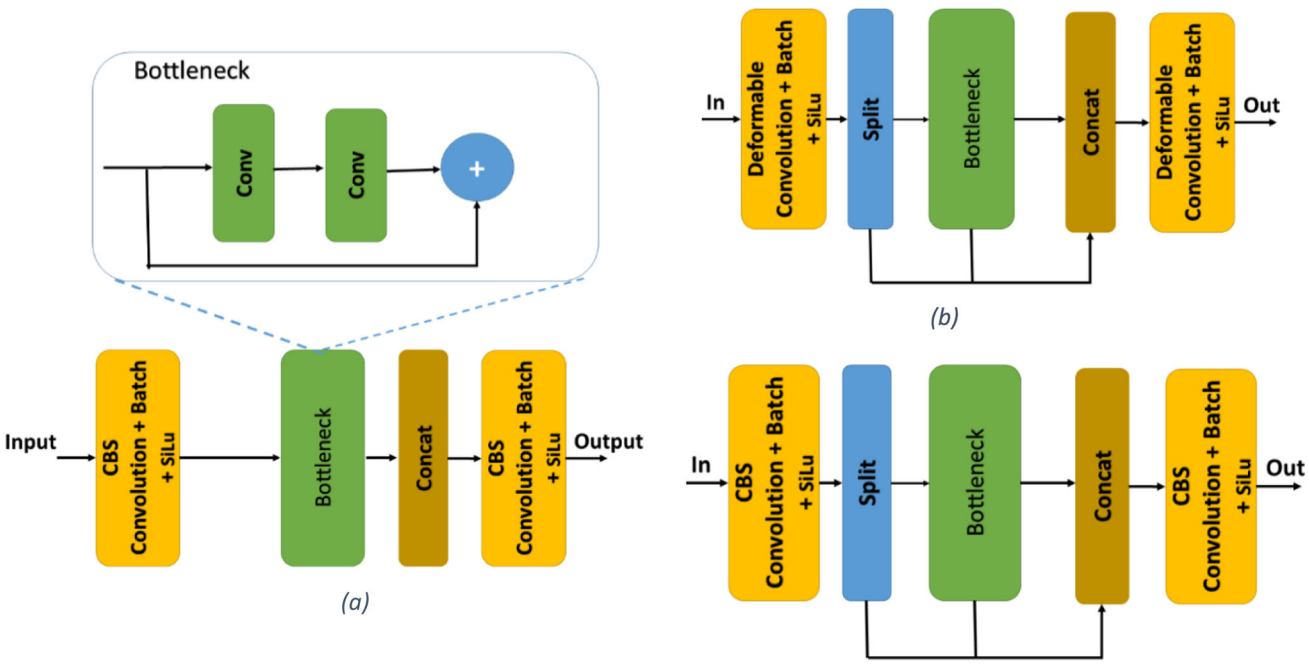


Fig. 9 The construction of C2, C2f, and D2f modules. **a** C2 module, **b** C2f module, **c** D2f module

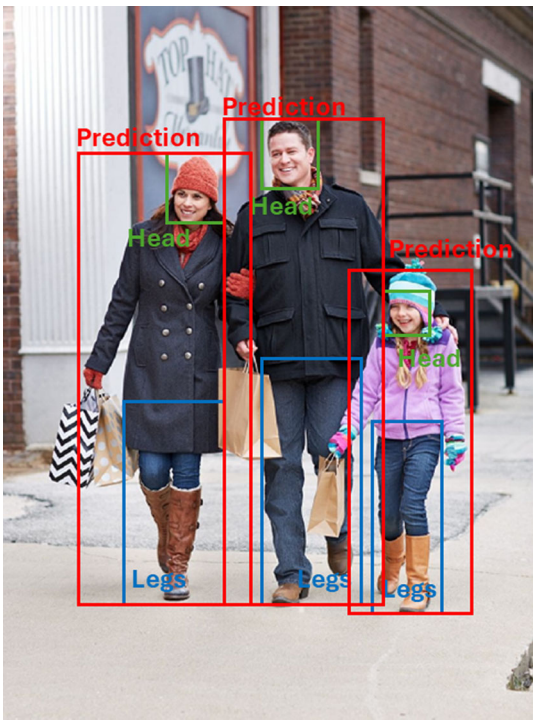


Fig. 10 The construction of full body bounding box from the face and leg bounding boxes

$$W = \frac{4}{3} W_{\text{leg}}, \; H = 2H_{\text{leg}} \tag{1}$$

where the head and leg regions' center point coordinates are denoted by xchead, ychead, xcleg, and ycleg. The coordinates of the pedestrian center point and the pedestrian's height and breadth are represented by xc, yc, H, and W, while Hhead, Whead, Hleg, and Wleg stand for the head and leg areas' height and width, respectively.

The resorted head and leg boxes are fused with the YOLO model predictions to illuminate the duplicate detection as shown in Fig. 11. Every pedestrian overall box recovered from the head region, every pedestrian

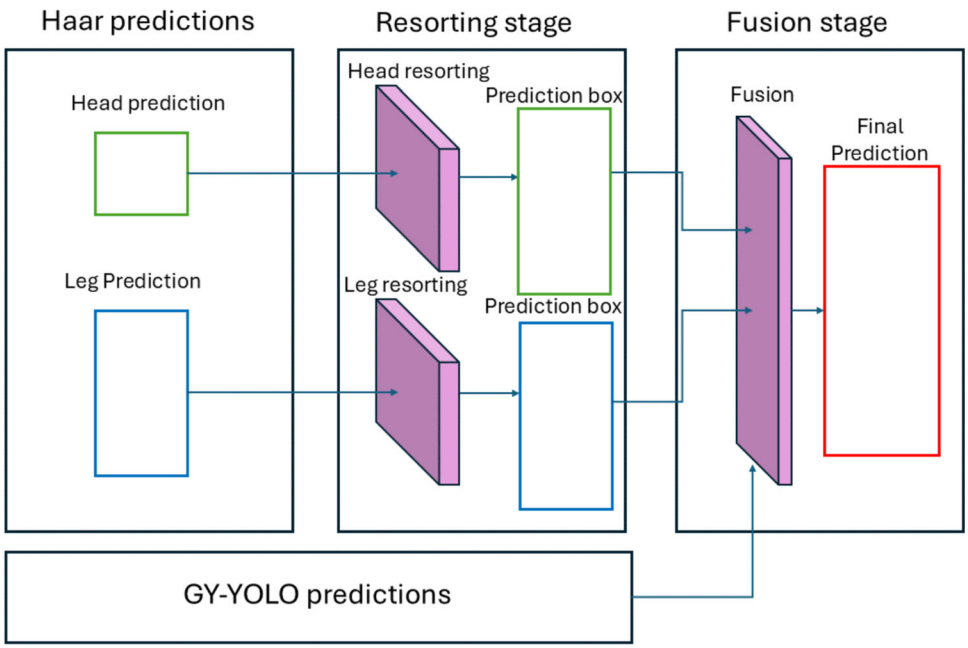


Fig. 11 Haar Cascade Fusion Module

overall box restored from the leg area, and every pedestrian overall box restored from the YOLO model will be filtered according to a certain IoU threshold. The whole architecture of the GS-YOLO model is shown in Fig. 12.

3.4 Preprocessing

The HEP dataset is derived from a mix of mobile camera footage and web-sourced images. This hybrid nature introduces issues with image resolution, angle, and consistency. Thus, a preprocessing stage is required to handle this issue. A list of the preprocessing techniques will be listed in this section.

3.4.1 Image resizing

To ensure uniform input dimensions, all images are adjusted to a fixed resolution (512 × 512 pixels). When scaling an image, padding which is adding borders to the original image helps it keep its aspect ratio. This avoids distortion when resizing pictures that aren't square.

3.4.2 Normalization

Normalization has the advantage of accelerating training convergence and lessening the impact of different image intensities. Standardization or (Z-Score Normalization) scales data to have zero mean and unit variance according to Eq. 2.

$$I_{\text{normalized}} = \frac{I - \mu}{\sigma} \tag{2}$$

where

I: is the original image's pixel intensity.
σ: is the standard deviation of the pixel intensities.
μ: is the mean of the pixel intensities.

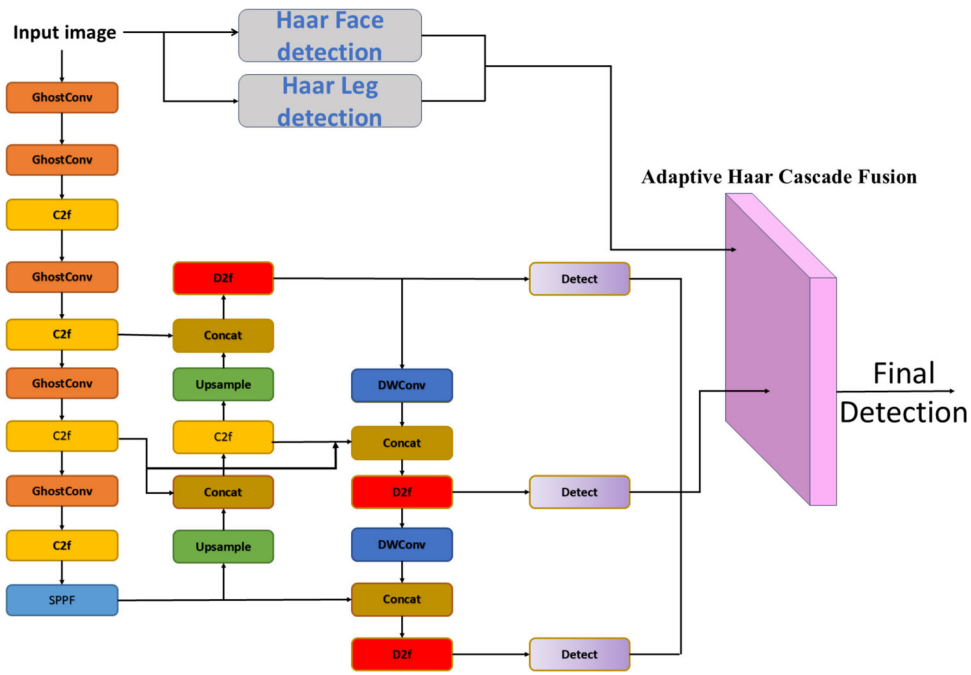


Fig. 12 GY-YOLO architecture

3.4.3 Data augmentation

To enrich the dataset, several augmentation techniques are applied, such as picture rotation, translation, scale, shear, flip left–right, and flip up-down. These geometric transformation techniques reduce the variation in orientations or angles. Another data augmentation method called CutMix [52] is adopted to increase the models' generalization and robustness. Successfully combining the content and supervision signals of two images and their accompanying labels improves training. Using CutMix data enhancement as a guide, mosaic data enhancement randomly scales, crops, and arranges four photos to create a new image [52]. Improving mosaic data can save costs, increase its value, strengthen the network's resilience, and need fewer graphics processing units (GPUs). The mathematical equation for CutMix is presented in Eqs. 3 and 4. A sample of this technique in training is shown in Fig. 13.

$$x_{\text{new}} = M \odot x_A + (1 - M) \odot xB \tag{3}$$

$$y_{\text{new}} = \lambda y_A + (1 - \lambda) yB \tag{4}$$

where

M: is a conversion of the patch area in binary.

$\odot$: is element-wise multiplication. $\lambda = \frac{\text{patch area}}{\text{image area}}$.



Fig. 13 Sample of the augmented images using CutMix technique

Table 2 Volume and density for the training subset of the proposed dataset and the different pedestrian detection datasets

	KITTI	CityPersons	CrowdHuman	HEP
#images	3,712	2,975	15,000	7136
#persons	2,322	19,238	3,39,565	40,482
#person/image	0.63	6.47	22.64	5.67

x_A and y_A are the image and label for the first sample.
xB and yB are the image and label for the second sample.

4 Simulation results and discussion

4.1 Data collection

The HEP dataset (Hybrid Egyptian Pedestrian dataset) is collected by mobile cameras for crowded crosses in different cities in Egypt such as Alexandria, Mansoura, and Cairo. Also, images from the web of people in traffic scenes are fed to the dataset for variation purposes. The collected images have diverse resolutions, the view angle (images captured from a camera installed in a moving vehicle and a mobile camera from the roadside), and the image sources (live cross intersections in different cities in Egypt and web images). The whole dataset images are for pedestrian scenarios which makes this dataset the most applicable and reliable training dataset for pedestrian detection tasks.

4.1.1 Dataset size

The collected dataset consists of 8920 images divided into 80% training (7136 images) with 40,482 persons in total and 20% validation (1784 images) with 4050 persons. The training images in the HEP dataset are 5 × more than the training data in the CityPersons [53] dataset and 6 × more than the KITTI [54] dataset images as listed in Table 2. However, the training data size in the HEP dataset is half of the images in the CrowdHuman [5], but the HEP dataset is the more efficient and reliable dataset for the pedestrian detection task.

4.1.2 Image annotation

Given that YOLO employs supervised learning, labeled datasets are required. An image annotation occurs when a label is applied to a group of images, as in segmentation, object identification, or image classification tasks. Annotations for object detection differ from picture classification in that multiple objects can appear in a single image. For every object, the matching class and the target bounding box—which indicates the object's location inside the image—must be obtained.

As shown in Fig. 14, the dataset is annotated with the YOLO format using the LabelImg application [55]. YOLO annotation is used to indicate the labels in a text file that shares the same name as the image. Following the matching class in the ground truth prediction are the bounding box dimensions, which include width, height, and X and Y coordinates for the object center. All of the numbers have been adjusted to match the image's height and width.

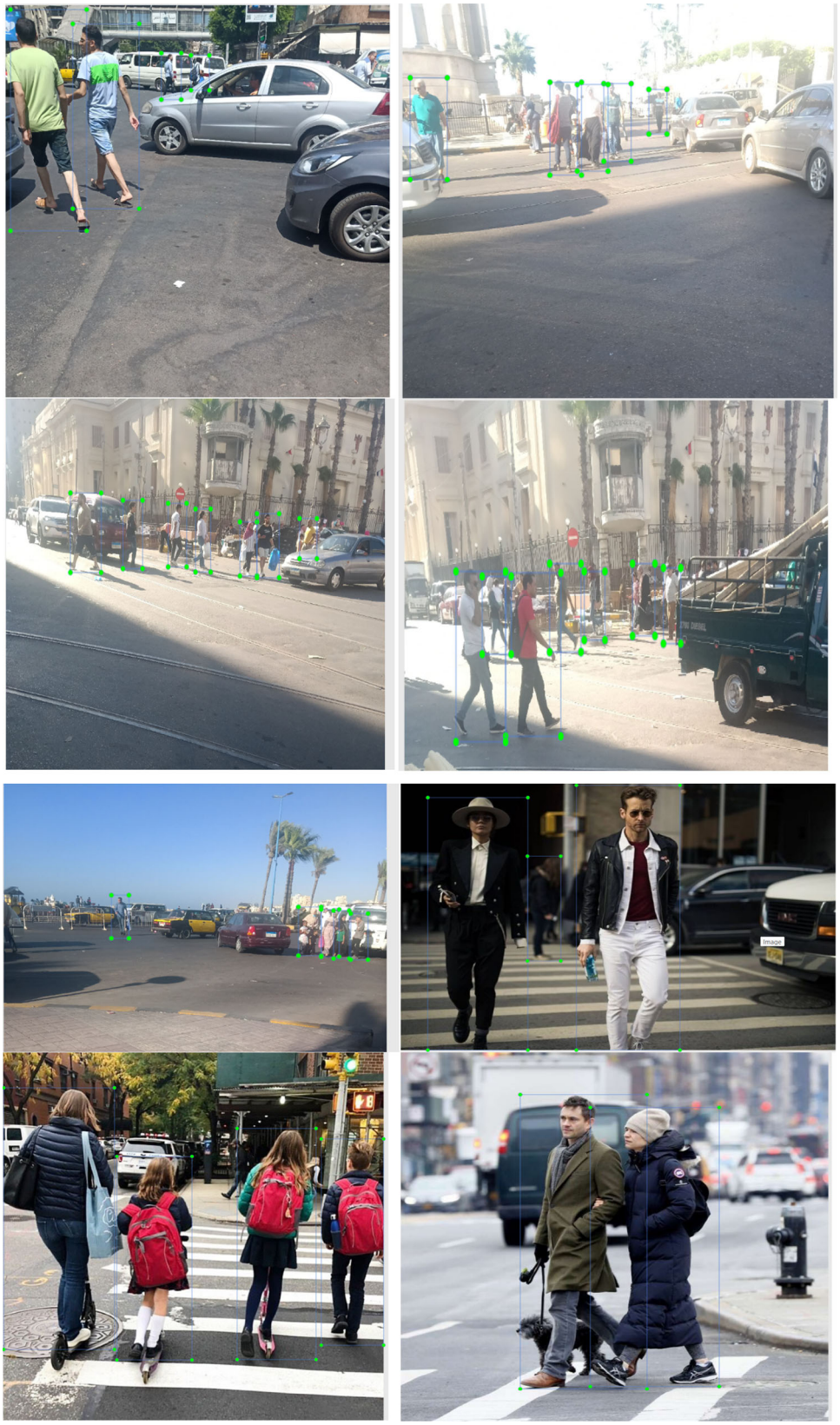


Fig. 14 Samples of the collected dataset

4.2 Simulation analysis and results

4.2.1 Evaluation metric

The models' mean average precision (mAP), recall, and precision are their performance indicators. Equation (5) defines precision as the ratio of real positive samples or samples that were correctly identified as objects, to total positive samples. The ratio of true positive samples—those that were correctly classified as objects—to correctly identified samples is known as the recall, and it is shown in Eq. (6). The F1 Score uses the algorithm to harmonize these measurements as formulated in Eq. (7). The F1 Score is significantly impacted by the values of true positives, false positives, and false negatives. A lower F1 Score would result from an increase in false negatives decreasing recall and an increase in false positives decreasing precision. On the other hand, increasing the number of true positives improves memory and precision, increasing the F1 Score.

$$\text{Precison} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{5}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{6}$$

$$F1 = \frac{2 * \text{precision} * \text{recall}}{\text{precision} + \text{recall}} \tag{7}$$

Average Precision is calculated as the weighted mean of precisions at each threshold, while the Mean Average of Precision (mAP) is the average of AP of each class as shown in Eq. (8).

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} AP_i \tag{8}$$

where N is the number of the classes, and AP_i is the average precision of the class i.

The trade-off between recall and precision is taken into account by the mAP, which also takes false positives (FP) and false negatives (FN) into account. Because of this feature, mAP is a useful statistic for the majority of detection applications. Mean average precision is calculated on different Intersection over Union (IoU). MAp50 is calculated at IOU = 0.5 and mAp50-95 is calculated over different IoU thresholds, from 0.5 to 0.95, step 0.05.

4.2.2 Specification for experiments

The experiments in this study are implemented on a Windows 11 using an Intel Core i7 CPU and an RTX 2060 GPU with 6GB VRAM. PyTorch 1.10 is the deep learning framework used in the development, and the experiments are carried out on two batches of training are carried out at a single, 640 × 640 image scale. The stochastic gradient descent (SGD) optimizer was employed throughout the training phase with a learning rate of 0.01 and a momentum of 0.937. All of the networks in the tests are trained from scratch without utilizing the official pre-training weights to provide a fair comparison for 100 epochs.

4.2.3 Results on HEP dataset

The dataset is trained for 100 epochs on YOLOv5 [13] and YOLOv8 [31] models. The first stage in the training process is to assign a patch of images from the data loader to the GPU to accelerate it. After that, the picture patch is distributed throughout the network, and the computations for classification and localization loss are conducted.

YOLO versions with varying scaling-up architectures in terms of parameters, floating-point operations (FLOPs), and number of layers demonstrate competitive performance after training on the gathered dataset. Table 1 shows comparative results between the different YOLO models. YOLOv5 and YOLOv8 come with five different variants that vary in the number of layers, parameters, and floating point operations. As the inference time is important, the smallest and most efficient variants of YOLO5 and YOLOv8 are chosen which are YOLOV5/8s and YOLOv5/8n.

YOLOv5 [12] seems to achieve better results than YOLOv8 [21] where 88.4% and 80.3% mAp with a 0.5 threshold is achieved for YOLOv5n and YOLOv5s, respectively, while YOLOv8s and YOLOv8n achieve 83.0% and 91.1% mAp with a 0.5 threshold, respectively. With a significant increase in the number of layers and parameters—as listed in Table 2, YOLOv8m appears to achieve better results. YOLOv8m is used as the base model for GS-YOLO, while GS-YOLO increases the mAp by 1%. GS-YOLO achieves 92.0% mAp on the collected dataset. Samples of the predicted output images are shown in Fig. 15.

To enhance precision and make the model more generalized to new data, k-fold cross-validation employs tenfold. Choosing a subset of k groups—in this case, 10 groups—as the test data and using the remaining data as training data is how k-fold cross-validation operates. Subsequently, the model is assessed in the chosen fold or group after being trained on the training subset. As shown in Table 3, GS-YOLO-10KF increases the mAp by 0.8% for the GS-YOLO model.

4.3 Discussion

The proposed dataset exhibits different challenges. The Scale of the pedestrians in the same image is not the same there could be some people close to the camera whereas others are far away. The brightness values are variable as the images are captured in different places and different conditions (day, night, and the sun is perpendicular to the face) over a long period. Also, there are situations where people could be wearing clothes similar to the background or the next car. There are other situations where the pedestrian is not looking at the camera (side view, back view, or looking at the phone). The dataset also contains images from very crowded places where people could persons cross the street between cars and not wait for the traffic signal or there is not a traffic signal. In such scenarios, occlusion can happen (people behind the car, people behind each other). Also as the data are collected from Arab countries, there are different dress codes such as hair and face cover, robe for men or different poses like a man holding a plastic bag in his hand, a woman holding a bag overhead, or a woman carrying a baby. These challenges and the proposed solution will be covered in this section.

4.3.1 Low-light detection

Pedestrian detection or object detection models face a significant challenge in accuracy due to the low lighting and brightness conditions. NightOwls [56] dataset suggests collecting data for only nighttime scenarios. However, none of the state-of-the-art pedestrian detection algorithms performs well at night detection. According to Xiao et al. [57], models can perform better in low-light conditions if trained using data from normal illumination. They discovered that features extracted by models trained under low-light conditions differ significantly from those trained under normal illumination, especially in terms of visual representations. Their model performed best when first pre-trained using standard lighting data and adjusted using data on low illumination. A recent study [58] discusses the appropriate balance of illumination in the training dataset. Their experiments utilize the Berkeley Deep-Drive 100K (BDD) dataset [59] which contains 80,000 images split into 42,000 for the day,

https://doi.org/10.1007/s00521-025-11207-4

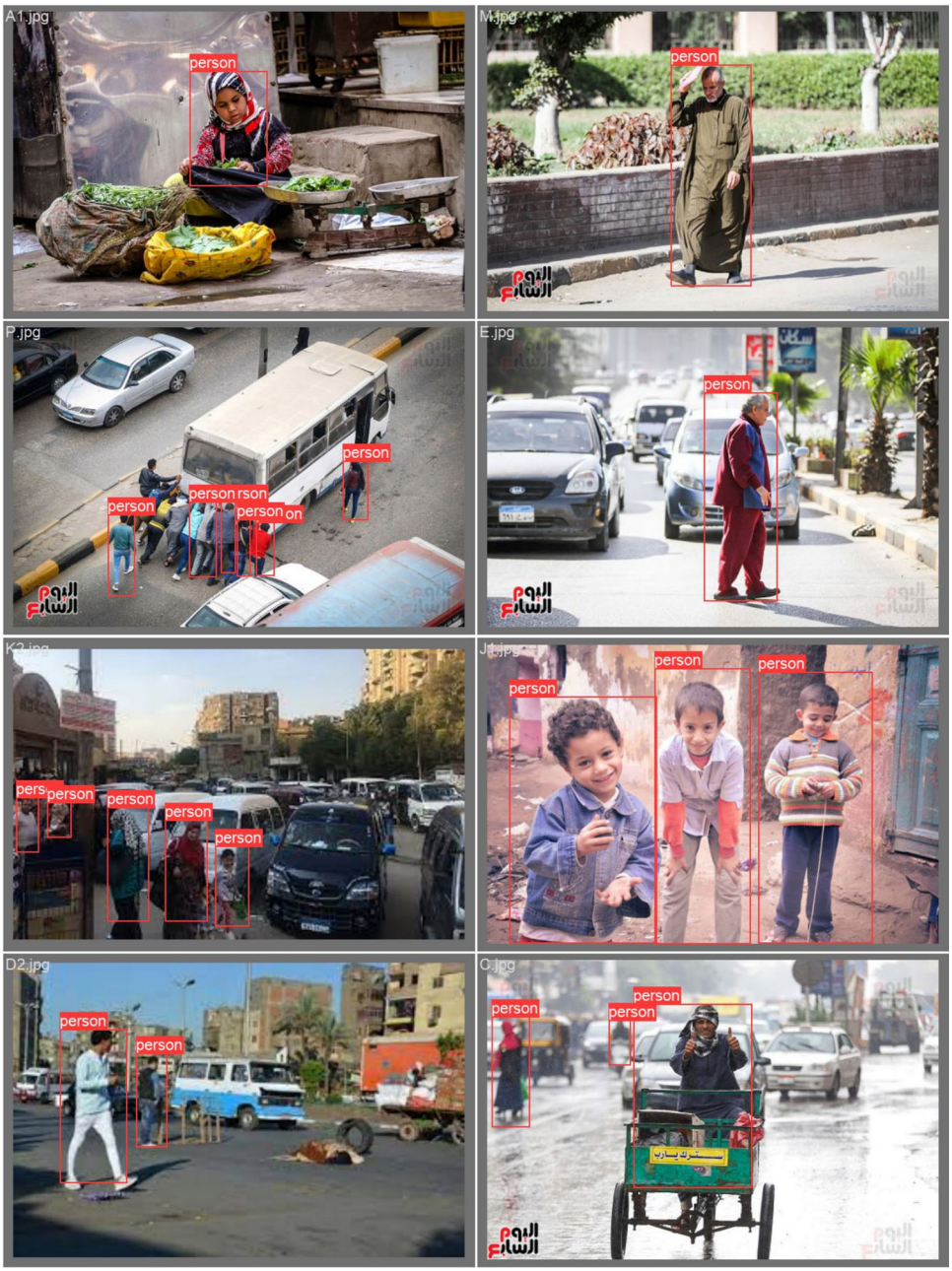


Fig. 15 Samples of the predicted results of GY-YOLO

32,000 for the night, and 6000 for dawn and dusk images. The training images are split into five different combinations which are D100N00 (100% daytime images and 0% nighttime images), D70N30, D50N50, D30N70, and D0N100. The results are evaluated using four categories of different light condition images (Day, Night, Mixed, and Dusk/Dawn) with 5000 images in each subset.

The study [58] concludes that daytime object detection performance will often improve as the proportion of training data collected during the day increases. The same outcome applies to data collected at night as well. A modest quantity of missing data will significantly improve the model's performance for the missing data category when it is first trained using only day or night data. The performance of the model is superior to that of other mixed training when it is trained using only one category of images (day or night). These recommendations are considered when collecting the HEP dataset which contains 30% of the nighttime images and 70% of daytime images. Moreover, we consider the HEP dataset to be more efficient and suitable for pedestrian detection tasks

Table 3 Comparative results between the different YOLOv5 and YOLOv8 models against the proposed GS-YOLO model on HEP dataset

	layers	Parameters	FLOPS	Precision	Recall	F1 score	mAp50	mAp50-95
YOLOv5n [13]	214	1.9 M	4.3 GFP	0.87	0.813	0.84	88.4	48
YOLOv5s [13]	212	7.2 M	15.9 GFP	0.884	0.813	0.847	90.3	53.5
YOLOv8n [31]	214	3.0 M	8.2GFP	0.832	0.748	0.787	83	43.1
YOLOv8s [31]	225	11.1 M	28.6 GFP	0.843	0.855	0.849	90.2	50.9
YOLOv8m [31]	295	25.9 M	79.3 GFP	0.856	0.86	0.857	91.1	52.9
GS-YOLO	307	30.8 M	110.1GFP	0.88	0.863	0.871	92	53.8
GS-YOLO-10KF	**307**	**30.8 M**	**110.1GFP**	**0.89**	**0.871**	**0.88**	**92.8**	**54.9**

Bold font indicates the best results among the compared methods

than the BDD dataset which contains fewer or zero pedestrians in nighttime images, while HEP is more specific and selective in collecting data that address the pedestrian scenarios. A sample of HEP dataset images compared to BDD images is visualized in Fig. 16.

Following the mentioned approach [58], the GY-YOLO performance is discussed along day, night, and image scenarios. The training data of 6000 images are split into different five categories as listed in Table 4. Where D100N00 means 100% of the images are daytime and 0% of nighttime images. The results are evaluated on three different subsets of images (i.e., day, night, and mixed) with 380 images in each subset. Table 5 shows the mAP (50) performance of the GY-YOLO for the different training image ratios.

4.3.2 Real-time detection

Due to the high computational load from deep learning models, achieving real-time pedestrian detection in fast-changing environments like traffic-heavy areas remains a significant challenge. Ghost modules can minimize floating point operations (FLOPs) and provide more feature maps with fewer parameters while maintaining accuracy.

An ablation tests are carried out to show the efficacy of the suggested GCL-YOLO network as listed in Table 6. The ablation experiments were designed as follows:

Ablation A1: Each convolution module in the backbone was replaced by a GhostConv module.

Ablation A2: Each convolution module in the head was replaced by a depthwise separable convolutions module.

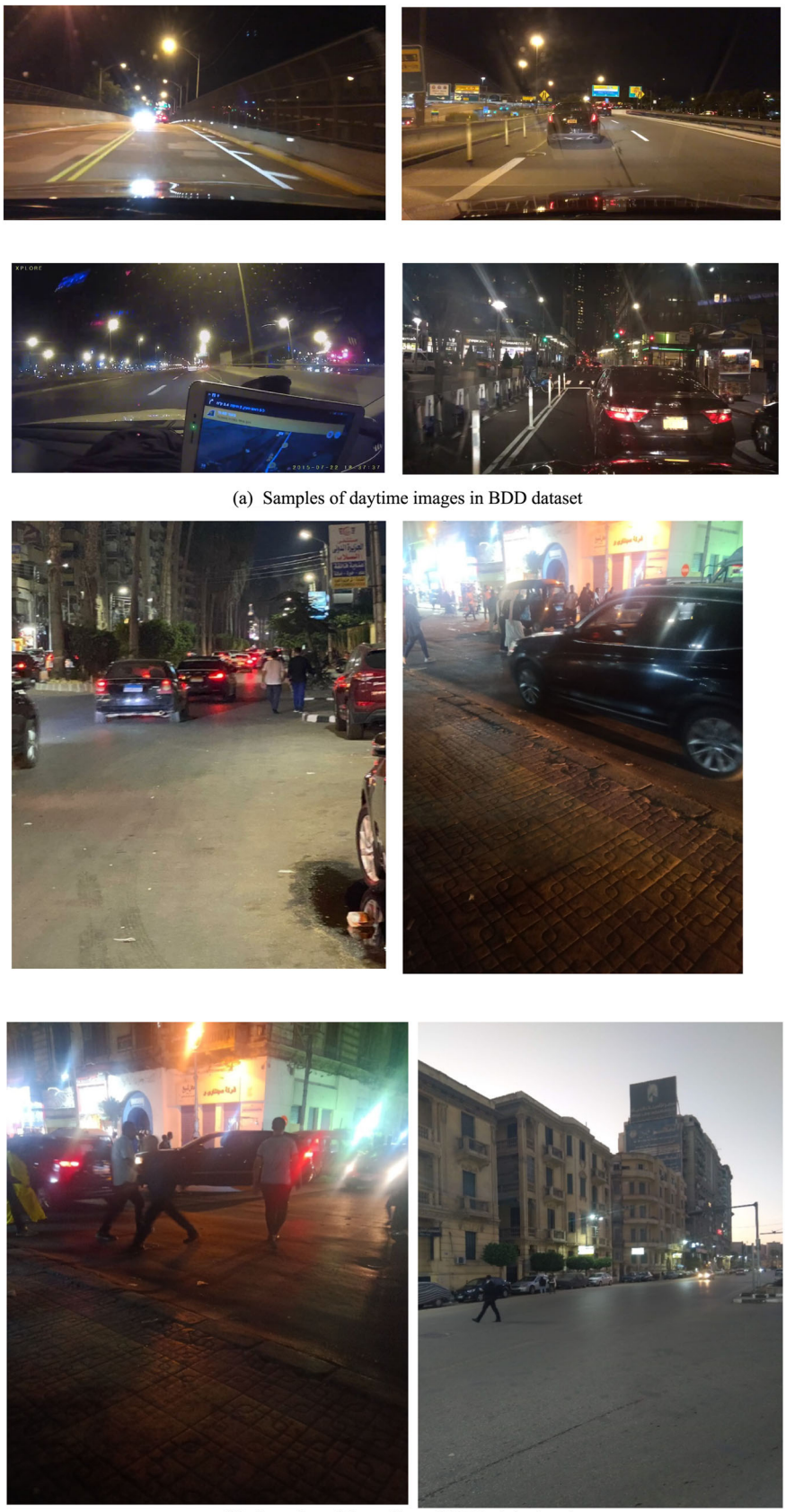
Ablation A3: Each C2f module in the head was replaced by D2f module.

As only half of the features in the GhostConv modules are employed to decrease the parameter and calculation quantities, directly replacing the convolution modules with the GhostConv modules in ablation A1 resulted in a degradation of detection accuracy.

Replacing each convolution module in the head with a depthwise separable convolution module also reduces the number of parameters and FLOPs with a higher gain than Ablation A1, but still lacks higher accuracy. Combining Ablation A1 and A2 achieves a competitive result with the original YOLOv8m with an efficient number of parameters and FLOPS. Emerging the D2f module rather than C2f achieves better results than YOLOv8m, but with more parameters and FLOPS due to the additional convolutional layers. These additional convolutional layers are learned by additional offsets from the previous feature maps.

Numerous hardware setups are used to test the real-time performance of the proposed GS-YOLO model. The RTX2060 GPU, the Intel Core i7 processor running at 2.5 GHz, the Jetson Nano Kit with the NVIDIA Maxwell architecture 128 core, and the Raspberry Pi 4 (4GB) are used to test the model inference speed. Both image and live video streams are used to test the model.

https://doi.org/10.1007/s00521-025-11207-4



(a) Samples of daytime images in BDD dataset

(b) Samples of the collected HEP nighttime images

Fig. 16 Comparison of the BDD nighttime images and the HEP nighttime images

Table 4 Training image ratio in the five different split

Ratio name	Day percentage	Night percentage	Image count
D100N00	100	0	6000
D70N30	70	30	6000
D50N50	50	50	6000
D30N70	30	70	6000
D00N100	0	0	6000

Table 5 mAP(50) performance of the GS-YOLO on the different data mixtures

Training data			Evaluation results (mAp (50))		
Ratio	Day	Night	Day	Night	Mixed
D100N00	6000	0	85.6	80.3	90.9
D70N30	4200	1800	87.5	82.4	91.2
D50N50	3000	3000	88.1	84.7	92
D30N70	1800	4200	83.5	84.1	91.3
D00N100	0	6000	79.2	83.9	90.1

Table 6 Comparison of inference speed on different architectures

Architecture	FPS
RTX2060 GPU	41
Intel core i7 @2.6 GHZ	28
Jetson nano GPU @ 128 cores	32
Raspberry Pi 4	23

GS-YOLO's inference performance is 41 frames per second (FPS) on the RTX 2060 GPU, 28 FPS on an Intel Core i7 processor running at 2.6 GHz, 32 FPS on a Jetson Nano GPU with 128 cores, and 23 FPS on Raspberry Pi 4. Table 7 presents the outcomes that have been reported.

5 Future work

To incorporate a greater range of lighting conditions, weather, and traffic patterns, we intend to grow the dataset by gathering photos from various geographic locations, seasons, and times of day. This would make it easier for the model to adjust to a variety of settings, including rural, suburban, and urban ones.

To ensure a more comprehensive portrayal of pedestrians and traffic conditions, we intend to increase the dataset size by incorporating more data from a variety of sources, such as open-source datasets or crowdsourced contributions.

This work can be extended to include much more data rather than from crowded streets in different countries or using Generative Adversarial Networks (GAN) to create similar situations for pedestrians. GS-YOLO can be

Table 7 Comparisons between the GY-YOLO model and the YOLOv8m model applying different combination of modifications

Model	A1	A2	A3	Params (M)	GFLOPs	mAp50(%)	FPS
YOLOv8m				25.9 M	79.3 GFP	91.1	
GS-YOLO	✔			22.8 M	58.3GFP	88.9	
		✔		20.2 M	40.8GFP	89.3	
	✔	✔		24.4 M	73.3GFP	90.5	
	✔	✔	✔	30.8 M	110.1 GFP	92	

modified by using one of the network pruning methods or using Neural Architecture Search (NAS) to search for the optimal backbone or head architecture for the pedestrian task.

6 Conclusions

This research presents a novel hybrid pedestrian dataset created by gathering web photos of individuals in traffic scenes and mobile camera photographs of people at pedestrian crossings in various Egyptian cities. Next, the dataset is labeled using the LabelImg tool for training purposes. Several augmentation strategies are applied to increase the size of the dataset that is delivered. The collected dataset is trained over YOLOv5 and YOLOv8 models and achieves promising results. GS-YOLO which includes Ghost and depth separable convolution layers instead of normal convolution layers in the backbone and the head, respectively, enhances the real-time performance. The Haar Cascade Fusion (HCF) module is proposed to overcome the occlusion issue by combining the head and leg features extracted from the Haar Cascade classifier. GS-YOLO also addresses the low-light detection problem by training the model using mixed data from day and night images. GS-YOLO achieves competitive results compared with the state-of-the-art models YOLOv5 and YOLOv8.

Author contributions Ali M. Elhenidy, Labib M. Labib, Amira Y. Haikal, and Mahmoud M. Saafan contributed to the design and implementation of the research and the analysis of the results. All the authors have participated in writing the manuscript and have revised the final version. All authors read and approved the final manuscript.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB). No funding was received for this work.

Data availability Data are Available upon request.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Elhenidy A, Mohamed L, Yassien A, Saafan M (2024) ScaledDETR: An alight weight object detection model for autonomous driving. Mansoura Eng J 49(5):13
2. Luo Y, Yin D, Wang A, Wu W (2018) Pedestrian tracking in surveillance video based on modified CNN. Multimed Tools Appl 77:24041–24058
3. Li M-L, Sun G-B, Yu J-X (2023) A pedestrian detection network model based on improved YOLOv5. Entropy 25:381. https://doi.org/10.3390/e25020381
4. Hasan I, Liao S, Li J, Ullah Akram S, and Shao L (2021) Generalizable pedestrian detection: the elephant in the room. In: IEEE/CVF conference on computer vision and pattern recognition (CVPR)
5. Shao S, Zhao Z, Li B, Xiao T, Yu G, Zhang X, and Sun J (2018) Crowdhuman: a benchmark for detecting human in a crowd. arXiv preprint arXiv:1805.00123

6. Zhang S, Xie Y, Wan J, Xia H, Li SZ, and Guo G (2019) Widerperson: a diverse dataset for dense pedestrian detection in the wild. IEEE transactions on multimedia
7. Wider Pedestrian (2019) competitions.codalab.org/competitions/20132
8. Pang Y, Cao J, Li Y, Xie J, Sun H, Gong J (2021) TJU-DHD: a diverse high-resolution dataset for object detection. IEEE Trans Image Process 30:207–219. https://doi.org/10.1109/TIP.2020.3034487
9. Carion N, Massa F, Synnaeve G, Usunier N, Kirillov A, and Zagoruyko S (2020) End-toend object detection with transformers. In: Proceedings Europe conference computer vision
10. Zhu X, Su W, Lu L, Li B, Wang X and Dai J (2020) Deformable DETR: deformable transformers for end-to-end object detection. In: Proceedings international conference learning representations
11. Lin M (2020) DETR for Crowd Pedestrian Detection. arXiv e-prints. https://doi.org/10.48550/arXiv.2012.06785.
12. Kyrkou C (2020) YOLOpeds: efficient real-time single-shot pedestrian detection for smart camera applications. IET Comput Vis 14:417–425. https://doi.org/10.1049/iet-cvi.2019.0897
13. Gu ZC, Zhu K, You ST (2023) YOLO-SSFS: a method combining SPD-Conv/STDL/IM-FPN/SIoU for outdoor small target vehicle detection. Electronics 12:3744
14. Hsu W-Y, Lin W-Y (2021) Ratio-and-scale-aware YOLO for pedestrian detection. IEEE Trans Image Process 30:934–947. https://doi.org/10.1109/TIP.2020.3039574
15. Lou H, Duan X, Guo J, Liu H, Gu J, Bi L, Chen H (2023) DC-YOLOv8: small-size object detection algorithm based on camera sensor. Electronics 12:2323
16. Luo X et al. (2024) A lightweight YOLOv5-FFM model for occlusion pedestrian detection. arXiv preprint arXiv:2408.06633
17. Jocher G (2020) Yolov5. Documentation at: https://docs.ultralytics.com/
18. Jocher G and Chaurasia A and Qiu J (2023) Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics
19 Aziz L, Haji Salam MSB, Sheikh UU, Ayub S (2020) Exploring deep learning-based architecture, strategies, applications and current trends in generic object detection: a comprehensive review. IEEE Access 8:170461–170495. https://doi.org/10.1109/ACCESS.2020.3021508
20. Liu L, Ouyang W, Wang X et al (2020) Deep learning for generic object detection: a survey. Int J Comput Vis 128:261–318. https://doi.org/10.1007/s11263-019-01247-4
21 Canny J (1986) A computational approach to edge detection. IEEE Trans Pattern Anal Mach Intell 8(6):679–698. https://doi.org/10.1109/TPAMI.1986.4767851
22. Viola P and Jones M (2001) Rapid object detection using a boosted cascade of simple features. computer vision and pattern recognition. CVPR 2001. In: Proceedings of the 2001 IEEE computer society conference on, IEEE, vol. 1, pp I–I.
23. Dalal N and Triggs B (2005) Histograms of oriented gradients for human detection. Computer vision and pattern recognition. CVPR 2005. In: IEEE computer society conference on, vol. 1. IEEE, pp 886–893
24. Lowe DG (1999) Object recognition from local scale-invariant features. Computer vision. In: The proceedings of the seventh IEEE international conference on, vol. 2. IEEE, pp 1150–1157
25. Girshick R, Donahue J, Darrell T, and Malik J (2014) Rich feature hierarchies for accurate object detection and semantic segmentation. In: Proceedings of the IEEE conference on computer vision and pattern recognition
26. Van de Sande KE, Uijlings JR, Gevers T, and Smeulders AW (2011) Segmentation as selective search for object recognition. In: Computer vision (ICCV), 2011 IEEE international conference on IEEE, pp 1879–1886
27. He K, Zhang X, Ren S, and Sun J (2014) Spatial pyramid pooling in deep convolutional networks for visual recognition. In: European conference on computer vision. Springer
28. Girshick R (2015) Fast R-CNN. In: IEEE international conference on computer vision (ICCV)
29. Ren S, He K, Girshick R, and Sun J (2015) Faster r-cnn: towards real-time object detection with region proposal networks. In: Advances in neural information processing systems, pp 91–99
30. Liu W, Anguelov D, Erhan D, Szegedy C, Reed S, Fu C-Y, Berg AC (2016) 'SSD: single shot multibox detector.' In: Leibe B, Matas J, Sebe N, Welling M (eds) Computer vision—ECCV. Springer, Cham, Switzerland, pp 21–37
31. Redmon J, Divvala S, Girshick R, and Farhadi A (2016) You only look once: unified, real-time object detection. In: Proceedings IEEE conference computer vision pattern recognition, pp 779–788
32. Redmon J and Farhadi A (2017) YOLO9000: better, faster, stronger. In: Proceedings IEEE conference computer vision pattern recognition, pp 6517–6525
33. Redmon J and Farhadi A (2018) YOLOv3: an incremental improvement. arXiv:1804.02767. [Online]. Available: https://arxiv.org/abs/1804.02767
34. Bochkovskiy A et al. (2020) YOLOv4: optimal speed and accuracy of object detection. ArXiv abs/2004.10934
35. Law H, Deng J (2018) CornerNet: detecting objects as paired keypoints. In: ECCV
36. Duan K, Bai S, Xie L, Qi H, Huang Q, and Tian Q (2019) CenterNet: keypoint triplets for object detection. arXiv:1904.08189.
37. Zhang S, Wen L, Bian X, Lei Z, and Li SZ (2018) Single-shot refinement neural network for object detection. In: Proceedings IEEE conference computer vision pattern recognition, pp 4203–4212

38. Simonyan K and Zisserman A (2014) Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556
39. He K, Zhang X, Ren S, and Sun J (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition
40. Dai J, Qi H, Xiong Y, Li Y, Zhang G, Hu H, and Wei Y (2017) Deformable convolutional networks. In: 2017 IEEE international conference on computer vision (ICCV), Venice, Italy, pp 764–773. https://doi.org/10.1109/ICCV.2017.89.
41. Howard AG, Zhu M, Chen B, Kalenichenko D, Wang W, Weyand T, Andreetto M, and Adam H (2017) MobileNets: efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861
42. Han K, Wang Y, Tian Q, Guo J, Xu C, Xu C (2020) Ghostnet: more features from cheap operations. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, Seattle, WA, USA, 14–19, pp 1580–1589
43. Ioffe S and Szegedy C (2015) Batch normalization: accelerating deep network training by reducing internal covariate shift. https://doi.org/10.48550/arXiv.1502.03167.
44. Lin TY, Dollar P, Girshick RB, He K, Hariharan B, and Belongie SJ (2017) Feature pyramid networks for object detection. CVPR, vol. 1, no. 2, p 4
45. Kuznetsova A, Rom H, Alldrin N, Uijlings J, Krasin I, Pont-Tuset J, Kamali S, Popov S, Malloci M, Kolesnikov A, Duerig T, and Ferrari V (2018) The open images dataset v4: unified image classification, object detection, and visual relationship detection at scale. arXiv:1811.00982. [Online]. Available: http://arxiv.org/abs/1811.00982
46. Lin TY, Maire M, Belongie S, Hays J, Perona P, Amanan D, DollÆr P, and Zitnick CL (2014) Microsoft COCO: common objects in context. In: Computer vision—ECCV, Fleet D, Pajdla T, Schiele B, and Tuytelaars T, (Eds.) Cham, Switzerland: Springer, pp 740–755
47. Liu S, Qi L, Qin H, Shi J, and Jia J (2018) Path aggregation network for instance segmentation. CVPR
48. Rukhovich D, Sofiiuk K, Galeev D, Barinova O, Konushin A (2021) IterDet: iterative scheme for object detection in crowded environments. In: Torsello A, Rossi L, Pelillo M, Biggio B, Robles-Kelly A (Eds.), Structural, syntactic, and statistical pattern recognition. S+SSPR 2021. Lecture notes in computer science, vol 12644. Springer, Cham. https://doi.org/10.1007/978-3-030-73973-7_33
49. Zheng A, Zhang Y, Zhang X, Qi X and Sun J (2022) Progressive end-to-end object detection in crowded scenes. https://doi.org/10.48550/arXiv.2203.07669
50. Hsu W-Y, Lin W-Y (2021) Adaptive fusion of multi-scale YOLO for pedestrian detection. IEEE Access 9:110063–110073. https://doi.org/10.1109/ACCESS.2021.3102600
51. Liu Q, Ye H, Wang S, Xu Z (2024) YOLOv8-CB: dense pedestrian detection algorithm based on in-vehicle camera. Electronics 13:236. https://doi.org/10.3390/electronics13010236
52. Yun S, Han D, Oh SJ et al. (2019) CutMix: regularization strategy to train strong classifiers with localizable features. In: IEEE international conference on computer vision 6022–6031
53. Zhang S et al. (2017) CityPersons: a diverse dataset for pedestrian detection. In: 2017 IEEE conference on computer vision and pattern recognition (CVPR), pp 4457–4465.
54. Geigerand A, Lenzand P and Urtasun R (2012) Are we ready for autonomous driving? The kitti vision benchmark suite. In: Conference on computer vision and pattern recognition(CVPR).
55. Label tool: https://github.com/heartexlabs/labelImg
56. Neumann L, Karg M, Zhang S, Scharfenberger C, Piegert E, Mistr S, Prokofyeva O, Thiel R, Vedaldi A, Zisserman A et al. (2019) NightOwls: a pedestrians at night dataset. In: Proceedings of the computer vision—ACCV 2018; Jawahar CV, Li H, Mori G, Schindler K (Eds.), Springer: Cham, Switzerland, pp 691–705
57. Xiao Y, Jiang A, Ye J, Wang MW (2020) Making of night vision: object detection under low-illumination. IEEE Access 8:123075–123086
58. Lin H, Parsi A, Mullins D, Horgan J, Ward E, Eising C, Denny P, Deegan B, Glavin M, Jones E (2024) A study on data selection for object detection in various lighting conditions for autonomous vehicles. J Imag 10:153. https://doi.org/10.3390/jimaging10070153
59. Yu F, Chen H, Wang X, Xian W, Chen Y, Liu F, Madhavan V, Darrell T (2020) BDD100K: a diverse driving dataset for heterogeneous multitask learning. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition, Seattle, WA, USA, 13–19, pp 2633–2642

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Ali M. Elhenidy[1,2] · Labib M. Labib[1] · Amira Y. Haikal[1] · Mahmoud M. Saafan[1,2]

✉ Mahmoud M. Saafan
saafan2007@mans.edu.eg

Ali M. Elhenidy
alielhenidy@mans.edu.eg

Labib M. Labib
labib_essa@mans.edu.eg

Amira Y. Haikal
amirayh@mans.edu.eg

[1] Computers and Control Systems Engineering Department, Faculty of Engineering, Mansoura University, Mansoura, Egypt

[2] Faculty of Engineering, Mansoura National University, Gamasa, Egypt